

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

«Реализация исторических алгоритмов шифрования»

Выполнил:

Суханкулиев Мухаммет,
студент группы N3346



(подпись)

Проверил:

Ищенко Алексей Петрович,
преподаватель, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Введение	3
1 Архитектура и реализация программного продукта	4
1.1 Структура проекта	4
1.2 Графический интерфейс и дополнительные возможности	4
2 Шифр №1: Шифр перестановки «Скитала»	6
2.1 Краткое описание алгоритма	6
2.2 Практическая реализация	6
2.3 Ограничения и криптостойкость	6
3 Шифр №2: «Шифровка решёткой» (Решётка Кардано)	7
3.1 Краткое описание алгоритма	7
3.2 Практическая реализация	7
3.3 Ограничения и криптостойкость	8
4 Руководство пользователя	9
4.1 Запуск и основной интерфейс	9
4.2 Шифрование/дешифрование	9
4.3 Использование дополнительных функций	9
5 Демонстрация работы	10
Заключение	12
Список использованных источников	13

ВВЕДЕНИЕ

Цель работы – разработка и исследование двух исторических алгоритмов перестановочного шифрования путем создания программного продукта с полнофункциональным графическим интерфейсом.

Для достижения поставленной цели необходимо решить следующие задачи:

- Реализовать алгоритмы шифрования и дешифрования для шифра Скитала и Решётки Кардано.
- Разработать интуитивно понятный графический пользовательский интерфейс (GUI) для взаимодействия с программой.
- Реализовать вспомогательные функции, требуемые техническим заданием: работа с файлами, загрузка встроенных примеров и отображение справочной информации по алгоритмам.
- Добавить дополнительные "креативные" возможности для повышения наглядности и удобства использования программы, такие как визуализация процессов шифрования.

1 АРХИТЕКТУРА И РЕАЛИЗАЦИЯ ПРОГРАММНОГО ПРОДУКТА

1.1 Структура проекта

Программа реализована на языке Python 3 и сознательно разделена на два логических модуля для соблюдения принципа разделения ответственности (Separation of Concerns):

- `ciphers.py`: Модуль бизнес-логики. Содержит только «чистые» функции для выполнения криптографических преобразований (шифрование, дешифрование) и генерации ключей (решётки). Этот модуль не зависит от способа представления данных и может быть переиспользован в любом другом приложении (например, консольном или веб).
- `app.py`: Главный модуль приложения. Отвечает за создание и управление всем графическим интерфейсом на базе стандартной библиотеки `tkinter`, обработку событий от пользователя (нажатия кнопок, ввод текста) и вызов соответствующих функций из модуля `ciphers.py`. Является точкой входа в программу.

Такая архитектура упрощает разработку, тестирование и дальнейшую поддержку кода.

1.2 Графический интерфейс и дополнительные возможности

Графический интерфейс (GUI) реализован с помощью стандартной библиотеки `tkinter` без использования внешних зависимостей для обеспечения максимальной портируемости. Для улучшения пользовательского опыта были реализованы следующие функции:

- **Темная тема:** Вместо стандартной системной темы реализована собственная неконтрастная темная тема. Это достигнуто путем детальной настройки стилей виджетов `ttk` с помощью класса `ttk.Style` и прямого конфигурирования виджетов `tk` (меню, текстовые поля) для полного контроля над цветовой палитрой.
- **Работа с файлами:** Через верхнее меню «Файл» пользователь может открывать текстовые файлы для шифрования и сохранять полученный результат, используя стандартные системные диалоги открытия/сохранения (`filedialog`).

- **Справка и примеры:** В меню «Справка» встроены окна с кратким описанием принципов работы, ключей и ограничений каждого алгоритма. Меню «Примеры» позволяет автоматически заполнить поля ввода демонстрационными данными.
- **Визуализация Решётки Кардано:** При каждой операции шифрования или дешифрования с помощью Решётки Кардано открывается дочернее окно, в котором наглядно отображается сгенерированная на основе ключа решётка-трафарет.
- **Анимация шифра Скитала:** Для наглядной демонстрации работы алгоритма реализовано отдельное окно анимации. Оно в пошаговом режиме, с подсветкой активной ячейки, показывает сначала процесс «записи» исходного текста в матрицу по строкам, а затем процесс «считывания» шифротекста по столбцам. Анимация реализована с помощью метода `self.after()` для исключения блокировки основного потока GUI.

2 ШИФР №1: ШИФР ПЕРЕСТАНОВКИ «СКИТАЛА»

2.1 Краткое описание алгоритма

Скитала — один из древнейших известных шифров перестановки, применявшийся в Древней Спарте. Принцип шифрования заключается в наматывании на цилиндрический стержень (скиталу) определенного диаметра узкой полоски пергамента. Сообщение записывается вдоль стержня по строкам. После разматывания ленты порядок букв на ней меняется, образуя шифротекст. Для расшифровки получателю необходима скитала точно такого же диаметра.

2.2 Практическая реализация

В программной реализации процесс имитируется с помощью двумерного массива (матрицы).

- **Ключ:** целое положительное число, соответствующее диаметру скиталы, т.е. количеству строк в матрице.

- **Шифрование:**

1. Вычисляется необходимое количество столбцов в матрице путем деления длины текста на ключ с округлением вверх (`math.ceil`).

2. Исходный текст дополняется пробелами до размера матрицы (метод `ljust`).

3. Текст последовательно записывается в матрицу по строкам (слева направо, сверху вниз).

4. Шифротекст считывается из матрицы по столбцам (сверху вниз, слева направо).

- **Дешифрование:** Процесс обратный. Шифротекст записывается в матрицу по столбцам, а исходный текст считывается по строкам. Лишние пробелы в конце удаляются.

2.3 Ограничения и криптостойкость

Шифр Скитала является простой моноалфавитной перестановкой. Его криптостойкость крайне низка. Он легко поддается взлому методом простого перебора ключа (достаточно попробовать намотать ленту на стержни нескольких разных диаметров). Частотный анализ символов также эффективен, так как частота встречаемости букв в шифротексте полностью сохраняется. Шифр имеет скорее историческую, чем практическую ценность.

3 ШИФР №2: «ШИФРОВКА РЕШЁТКОЙ» (РЕШЁТКА КАРДАНО)

3.1 Краткое описание алгоритма

Решётка Кардано — это шифр перестановки, использующий в качестве ключа трафарет (решётку) с вырезанными в нем ячейками. Трафарет обычно имеет квадратную форму. Шифрование происходит путем наложения решётки на лист бумаги, записи части сообщения в вырезы, поворота решётки на 90 градусов и повторения операции.

В данной реализации используется усовершенствованный подход, где решётка не хранится секретно, а генерируется на основе ключевого слова.

3.2 Практическая реализация

- **Ключ:** пара из строки (ключевое слово) и целого чётного числа (размер решётки).

- **Генерация решётки:** Это ключевой аспект реализации. Чтобы для одного и того же ключевого слова всегда генерировалась одна и та же решётка, перед генерацией инициализируется генератор псевдослучайных чисел (`random.seed(key_phrase)`). Сам алгоритм генерации гарантирует, что при четырёх поворотах вырезанные ячейки не накладываются друг на друга и покрывают всю площадь ровно один раз.

- **Шифрование:**

1. Исходный текст дополняется до размера решётки ($N \times N$) специальным символом ` \_ `.

2. Часть дополненного текста записывается в ячейки матрицы, соответствующие «отверстиям» в решётке.

3. Решётка мысленно поворачивается на 90 градусов по часовой стрелке.

4. Шаги 2 и 3 повторяются еще три раза, пока вся матрица не будет заполнена.

5. Символы из матрицы считываются последовательно, образуя шифротекст.

- **Дешифрование:** Процесс, обратный шифрованию. Для восстановления решётки используется то же ключевое слово и размер. Шифротекст преобразуется в матрицу, из которой символы считываются через отверстия поворачиваемой решётки. Символы ` \_ ` в конце удаляются.

3.3 Ограничения и криптостойкость

Данный шифр также является перестановочным. Его криптостойкость полностью зависит от секретности ключевого слова. Если злоумышленник подберет ключевое слово, он сможет восстановить решётку и расшифровать сообщение. Длина ключа (слова) напрямую влияет на сложность его подбора. Тем не менее, шифр уязвим для более сложных методов криптоанализа перестановочных шифров при наличии достаточного объема шифротекста.

4 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

4.1 Запуск и основной интерфейс

Программа запускается с помощью файла `app.py` (требуется Python) или скомпилированного `app.exe`. Главное окно состоит из следующих областей:

- **Область управления:** выбор одного из двух шифров.
- **Поле «Исходный текст»:** для ввода или вставки текста.
- **Панель действий:** кнопки «Шифровать», «Дешифровать», «Визуализировать» (для Скиталы) и «Очистить всё».
- **Панель ключей:** поля для ввода параметров шифрования, меняются в зависимости от выбранного алгоритма.
- **Поле «Результат»:** для отображения зашифрованного/дешифрованного текста.

4.2 Шифрование/дешифрование

1. Выберите нужный шифр (Скитала или Решётка Кардано).
2. Введите/вставьте текст в поле «Исходный текст».
3. Заполните параметры ключа (число для Скиталы; слово и размер для Решётки Кардано).
4. Нажмите «Шифровать» для получения шифротекста или «Дешифровать» для восстановления исходного текста. Результат появится в нижнем поле.

4.3 Использование дополнительных функций

- **Работа с файлами:** Используйте меню «Файл» -> «Открыть...» для загрузки текста из `.txt` файла и «Файл» -> «Сохранить как...» для сохранения результата.
- **Примеры:** Меню «Примеры» позволяет автоматически заполнить все поля корректными данными для демонстрации работы каждого шифра.
- **Справка:** Меню «Справка» открывает окна с теоретической информацией по каждому алгоритму.
- **Визуализация:** При выбранном шифре Скитала, после ввода текста и ключа, кнопка «Визуализировать» открывает окно с пошаговой анимацией процесса шифрования.

5 ДЕМОНСТРАЦИЯ РАБОТЫ

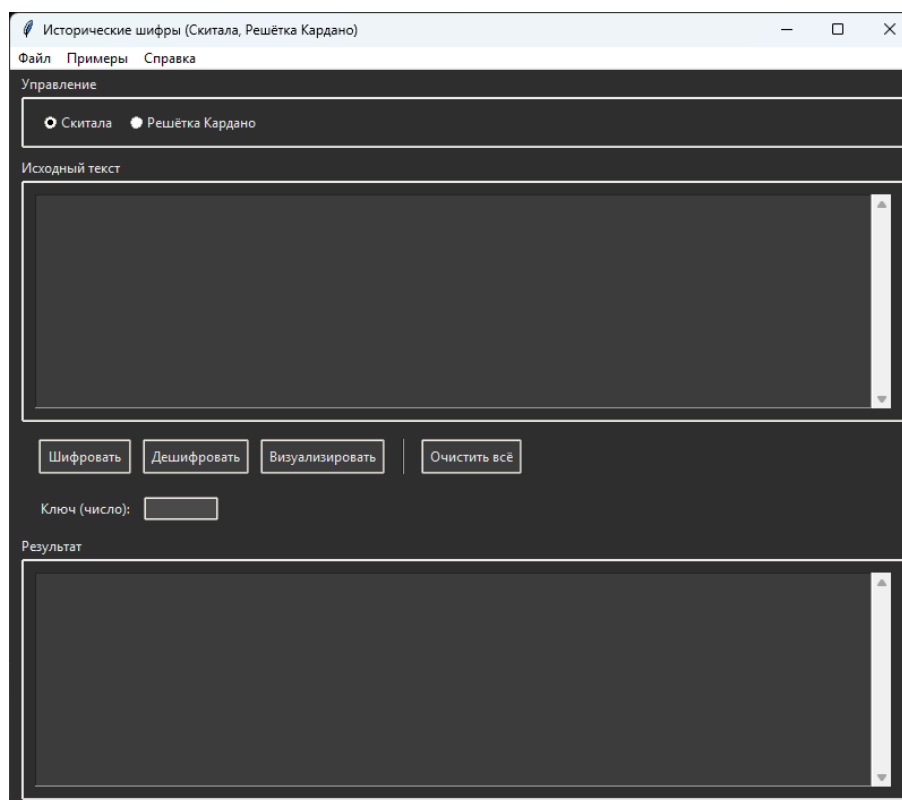


Рисунок 1 – Главное окно программы с темной темой

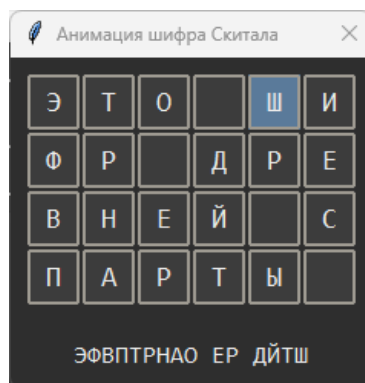


Рисунок 2 – Анимация процесса шифрования для шифра Скитала

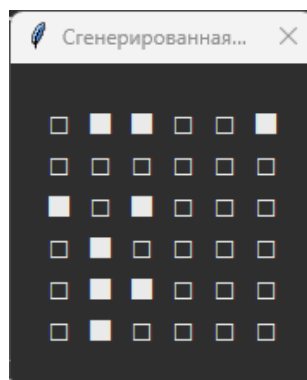


Рисунок 3 – Визуализация решётки Кардано, сгенерированной по ключу

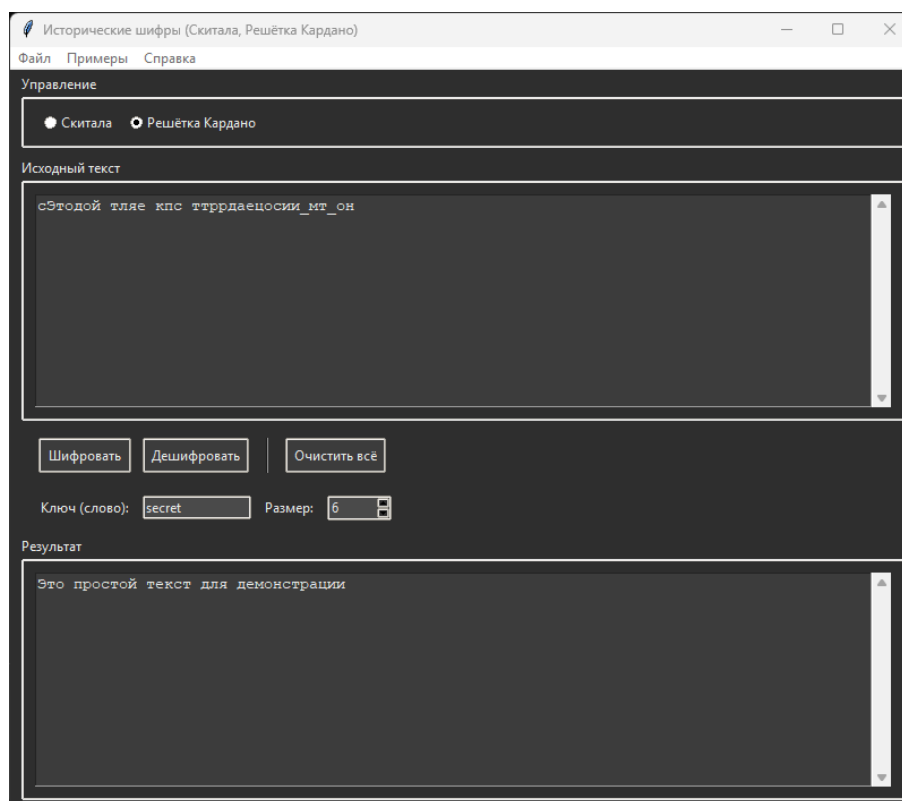


Рисунок 4 – Пример успешного цикла шифрования и дешифрования

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были успешно реализованы два исторических алгоритма перестановочного шифрования: шифр Скитала и Решётка Кардано. Разработан полнофункциональный программный продукт с графическим пользовательским интерфейсом на базе стандартной библиотеки tkinter.

Программа полностью соответствует техническому заданию, включая реализацию режимов шифрования и дешифрования, возможность работы с файлами, а также наличие встроенных примеров и справочной информации.

В качестве «креативной» составляющей были добавлены интерактивные элементы: наглядная пошаговая анимация для шифра Скитала и визуализация генерируемой решётки для шифра Кардано. Для улучшения пользовательского опыта была реализована собственная темная тема интерфейса.

Работа позволила закрепить навыки разработки GUI-приложений на Python, реализации криптографических алгоритмов и проектирования программных продуктов с разделением бизнес-логики и представления.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ищенко А. П. Технологии и методы программирования – Лекция 5: материалы лекции / А. П. Ищенко. – Санкт-Петербург : Университет ИТМО, 2025. – Текст : непосредственный.
2. Сингх С. Книга шифров. Тайная история шифров и их расшифровки / С. Сингх. – Москва : АСТ, 2007. – 448 с. – Текст : непосредственный.
3. Tkinter 8.5 reference: a GUI for Python : техническая документация. – Текст : электронный. – 2023. – URL: <https://tkdocs.com/>
4. Бабаш А. В., Шанкин Г. П. История криптографии. Часть I. — М.: Гелиос АРВ, 2002. — 240 с. — ISBN 5-85438-043-9.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг А.1 – ciphers.py

```
"""
Модуль, содержащий бизнес-логику алгоритмов шифрования.
"""

import math
import random
from typing import List, Tuple

# --- Шифр Скитала ---

def scytale_encrypt(text: str, key: int) -> str:
    """Шифрование текста методом Скиталы."""
    if not isinstance(key, int) or key <= 0:
        raise ValueError("Ключ должен быть положительным целым числом.")
    if not text:
        return ""
    num_cols = math.ceil(len(text) / key)
    padded_text = text.ljust(key * num_cols)
    matrix = ["" * num_cols for _ in range(key)]
    char_index = 0
    for row in range(key):
        for col in range(num_cols):
            matrix[row][col] = padded_text[char_index]
            char_index += 1
    ciphertext_parts = [matrix[r][c] for c in range(num_cols) for r in
range(key)]
    return "".join(ciphertext_parts)

def scytale_decrypt(ciphertext: str, key: int) -> str:
    """Дешифрование текста, зашифрованного методом Скиталы."""
    if not isinstance(key, int) or key <= 0:
        raise ValueError("Ключ должен быть положительным целым числом.")
    if not ciphertext:
        return ""
    num_cols = math.ceil(len(ciphertext) / key)
    num_rows = key
    matrix = ["" * num_cols for _ in range(num_rows)]
    char_index = 0
    for col in range(num_cols):
        for row in range(num_rows):
            if char_index < len(ciphertext):
                matrix[row][col] = ciphertext[char_index]
                char_index += 1
    plaintext_parts = [matrix[r][c] for r in range(num_rows) for c in
range(num_cols)]
    return "".join(plaintext_parts).rstrip()

# --- Решётка Кардано ---
```

```

def _rotate_grille_90_clockwise(grille: List[List[bool]]) ->
List[List[bool]]:
    """Вспомогательная функция для поворота решётки на 90 градусов."""
    size = len(grille)
    new_grille = [[False] * size for _ in range(size)]
    for r in range(size):
        for c in range(size):
            new_grille[c][size - 1 - r] = grille[r][c]
    return new_grille

def cardan_generate_grille(size: int) -> List[List[bool]]:
    """Генерирует валидную решётку Кардано заданного размера."""
    if not isinstance(size, int) or size <= 0 or size % 2 != 0:
        raise ValueError("Размер решётки должен быть чётным положительным
числом.")
    grille = [[False] * size for _ in range(size)]
    temp_matrix = [[False] * size for _ in range(size)]
    holes_to_make = (size * size) // 4
    hole_count = 0
    all_coords = [(r, c) for r in range(size) for c in range(size)]
    random.shuffle(all_coords)
    for r, c in all_coords:
        coords: List[Tuple[int, int]] = [
            (r, c),
            (c, size - 1 - r),
            (size - 1 - r, size - 1 - c),
            (size - 1 - c, r),
        ]
        if all(not temp_matrix[row][col] for row, col in coords):
            grille[r][c] = True
            hole_count += 1
            for row, col in coords:
                temp_matrix[row][col] = True
            if hole_count == holes_to_make:
                break
    return grille

def cardan_encrypt(text: str, grille: List[List[bool]]) -> str:
    """Шифрование текста с использованием решётки Кардано."""
    size = len(grille)
    if not all(len(row) == size for row in grille):
        raise ValueError("Решётка должна быть квадратной.")
    if len(text) != size * size:
        raise ValueError(f"Длина текста ({len(text)}) должна быть равна
{size*size}.")
    current_grille = grille
    result_matrix = [[""] * size for _ in range(size)]
    text_idx = 0
    for _ in range(4):
        for r in range(size):
            for c in range(size):
                if current_grille[r][c]:
                    result_matrix[r][c] = text[text_idx]
                    text_idx += 1
            current_grille = _rotate_grille_90_clockwise(current_grille)
    return "".join("".join(row) for row in result_matrix)

def cardan_decrypt(ciphertext: str, grille: List[List[bool]]) -> str:
    """Дешифрование текста, зашифрованного решёткой Кардано."""

```

```

size = len(grille)
if not all(len(row) == size for row in grille):
    raise ValueError("Решётка должна быть квадратной.")
if len(ciphertext) != size * size:
    raise ValueError(
        f"Длина шифротекста ({len(ciphertext)}) должна быть равна
{size*size}."
    )
cipher_matrix = ["" * size for _ in range(size)]
char_idx = 0
for r in range(size):
    for c in range(size):
        cipher_matrix[r][c] = ciphertext[char_idx]
        char_idx += 1
current_grille = grille
plaintext_parts = []
for _ in range(4):
    for r in range(size):
        for c in range(size):
            if current_grille[r][c]:
                plaintext_parts.append(cipher_matrix[r][c])
    current_grille = _rotate_grille_90_clockwise(current_grille)
return "".join(plaintext_parts)

```

Листинг A.2 – app.py

```

"""
Главный модуль приложения для шифрования историческими алгоритмами.
Содержит всю логику графического интерфейса и является точкой входа.
"""

import math
import random
import tkinter as tk
from tkinter import ttk, messagebox, scrolledtext, filedialog

import ciphers

# Класс для окна с анимацией шифра Скитала
class ScytaleAnimationWindow(tk.Toplevel):
    """Дочернее окно для визуализации процесса шифрования Скиталы."""

    def __init__(self, master, text, key, colors):
        super().__init__(master)
        self.title("Анимация шифра Скитала")
        self.transient(master)
        self.resizable(False, False)
        self.configure(bg=colors["bg"])

        self.ANIMATION_DELAY = 150
        self.HIGHLIGHT_COLOR = colors["highlight"]
        self.DEFAULT_COLOR = colors["widget_bg"]

        self.rows = key
        self.cols = math.ceil(len(text) / key)
        self.padded_text = text.ljust(self.rows * self.cols)

        self.labels = []
        self.result_text = tk.StringVar()

```



```

        style = ttk.Style(self)
        style.configure("TLabel", background=colors["bg"],
foreground=colors["fg"])
        style.configure(
            "Anim.TLabel", background=self.DEFAULT_COLOR,
foreground=colors["fg"]
        )

        grid_frame = ttk.Frame(self, padding=10)
        grid_frame.pack()
        for r in range(self.rows):
            row_labels = []
            for c in range(self.cols):
                label = ttk.Label(
                    grid_frame,
                    text=" ",
                    font=("Consolas", 14),
                    borderwidth=1,
                    relief="solid",
                    padding=5,
                    width=2,
                    anchor="center",
                    style="Anim.TLabel",
                )
                label.grid(row=r, column=c, padx=1, pady=1)
                row_labels.append(label)
            self.labels.append(row_labels)

        ttk.Label(self, textvariable=self.result_text, font=("Consolas",
12)).pack(
            pady=10
        )
        self.after(500, self.start_animation)

    def start_animation(self):
        self.current_pos = -1
        self.result_chars = []
        self._animate_write_step()

    def _animate_write_step(self):
        """Рекурсивный вызов с задержкой для пошаговой анимации записи."""
        if self.current_pos >= 0:
            prev_r, prev_c = divmod(self.current_pos, self.cols)
            self.labels[prev_r][prev_c].config(background=self.DEFAULT_COLOR)
            self.current_pos += 1
            if self.current_pos >= len(self.padded_text):
                self.current_pos = -1
                self.after(1000, self._animate_read_step)
            return
        r, c = divmod(self.current_pos, self.cols)
        self.labels[r][c].config(
            text=self.padded_text[self.current_pos],
background=self.HIGHLIGHT_COLOR
        )
        self.after(self.ANIMATION_DELAY, self._animate_write_step)

    def _animate_read_step(self):
        """Рекурсивный вызов с задержкой для пошаговой анимации чтения."""
        if self.current_pos >= 0:
            prev_c, prev_r = divmod(self.current_pos, self.rows)
            self.labels[prev_r][prev_c].config(background=self.DEFAULT_COLOR)
            self.current_pos += 1

```

```

        if self.current_pos >= self.rows * self.cols:
            self.result_text.set(self.result_text.get() + " -> Готово!")
            return
        c, r = divmod(self.current_pos, self.rows)
        char = self.labels[r][c].cget("text")
        self.result_chars.append(char)
        self.result_text.set("".join(self.result_chars))
        self.labels[r][c].config(background=self.HIGHLIGHT_COLOR)
        self.after(self.ANIMATION_DELAY, self._animate_read_step)

class CipherApp(tk.Tk):
    """Основной класс приложения с графическим интерфейсом."""

    def __init__(self):
        super().__init__()

        # --- Настройка темной темы вручную ---
        self.COLORS = {
            "bg": "#2E2E2E",
            "fg": "#EAEAEA",
            "widget_bg": "#3C3C3C",
            "highlight": "#5A7A9A",
            "entry_bg": "#4A4A4A",
        }
        self.configure(bg=self.COLORS["bg"])
        self.setup_styles()

        self.title("Исторические шифры (Скитала, Решётка Кардано)")
        self.geometry("800x650")
        self._create_widgets()
        self._create_context_menu()
        self._create_menu_bar()

    def setup_styles(self):
        """Настраивает стиль ttk виджетов для темной темы."""
        style = ttk.Style(self)
        style.theme_use("clam")

        style.configure(
            ".",
            background=self.COLORS["bg"],
            foreground=self.COLORS["fg"],
            fieldbackground=self.COLORS["widget_bg"],
            borderwidth=1,
        )

        style.configure(
            "TLabel", background=self.COLORS["bg"],
            foreground=self.COLORS["fg"]
        )

        style.configure(
            "TButton", background=self.COLORS["widget_bg"],
            foreground=self.COLORS["fg"]
        )

        style.map("TButton", background=[("active",
            self.COLORS["highlight"])]])

        style.configure(
            "TRadiobutton", background=self.COLORS["bg"],
            foreground=self.COLORS["fg"]
        )

```

```

        style.map("TRadiobutton", background=[("active",
self.COLORS["highlight"])]))
        style.configure(
            "TEntry",
            fieldbackground=self.COLORS["entry_bg"],
            foreground=self.COLORS["fg"],
            insertcolor=self.COLORS["fg"],
        )
        style.configure(
            "TSpinbox",
            fieldbackground=self.COLORS["entry_bg"],
            foreground=self.COLORS["fg"],
        )
        style.configure(
            "TLabelframe", background=self.COLORS["bg"],
bordercolor=self.COLORS["fg"]
        )
        style.configure(
            "TLabelframe.Label",
            background=self.COLORS["bg"],
            foreground=self.COLORS["fg"],
        )

        self.option_add("*TearOff", False)
        self.option_add("*Text*background", self.COLORS["widget_bg"])
        self.option_add("*Text*foreground", self.COLORS["fg"])
        self.option_add("*Text*insertBackground", self.COLORS["fg"])
        self.option_add("*Text*selectBackground", self.COLORS["highlight"])

def _create_menu_bar(self):
    """Создает и настраивает верхнее меню приложения."""
    main_menu = tk.Menu(
        self, bg=self.COLORS["widget_bg"], fg=self.COLORS["fg"], bd=0
    )
    self.config(menu=main_menu)

    file_menu = tk.Menu(
        main_menu, tearoff=0, bg=self.COLORS["widget_bg"],
fg=self.COLORS["fg"]
    )
    file_menu.add_command(
        label="Открыть текст из файла...", command=self._open_file
    )
    file_menu.add_command(
        label="Сохранить результат в файл...", command=self._save_file
    )
    file_menu.add_separator()
    file_menu.add_command(label="Выход", command=self.destroy)
    main_menu.add_cascade(label="Файл", menu=file_menu)

    examples_menu = tk.Menu(
        main_menu, tearoff=0, bg=self.COLORS["widget_bg"],
fg=self.COLORS["fg"]
    )
    examples_menu.add_command(
        label="Пример для Скиталы", command=self._load_scytale_example
    )
    examples_menu.add_command(
        label="Пример для Решётки Кардано",
command=self._load_cardan_example
    )
    main_menu.add_cascade(label="Примеры", menu=examples_menu)

```

```

        help_menu = tk.Menu(
            main_menu, tearoff=0, bg=self.COLORS["widget_bg"],
            fg=self.COLORS["fg"]
        )
        help_menu.add_command(label="О шифре Скитала",
            command=self._show_scytale_help)
        help_menu.add_command(label="О Решётке Кардано",
            command=self._show_cardan_help)
        main_menu.add_cascade(label="Справка", menu=help_menu)

    def _create_context_menu(self):
        # Привязываем меню к виджету в фокусе в момент клика.
        self.context_menu = tk.Menu(
            self, tearoff=0, bg=self.COLORS["widget_bg"],
            fg=self.COLORS["fg"]
        )
        self.context_menu.add_command(
            label="Копировать",
            command=lambda: self.focus_get().event_generate("<<Copy>>"),
        )
        self.context_menu.add_command(
            label="Вставить",
            command=lambda: self.focus_get().event_generate("<<Paste>>"),
        )
        self.context_menu.add_command(
            label="Вырезать", command=lambda:
self.focus_get().event_generate("<<Cut>>")
        )

    def _show_context_menu(self, event):
        widget = self.focus_get()
        if isinstance(widget, (tk.Text, scrolledtext.ScrolledText,
            ttk.Entry)):
            widget.bind(
                "<Button-3>", self.context_menu.tk_popup(event.x_root,
event.y_root)
            )

    def _create_widgets(self):
        self.selected_cipher = tk.StringVar(value="scytale")
        control_frame = ttk.LabelFrame(self, text="Управление", padding="10")
        control_frame.pack(fill=tk.X, padx=10, pady=5)
        ttk.Radiobutton(
            control_frame,
            text="Скитала",
            variable=self.selected_cipher,
            value="scytale",
            command=self._update_ui,
        ).pack(side=tk.LEFT, padx=5)
        ttk.Radiobutton(
            control_frame,
            text="Решётка Кардано",
            variable=self.selected_cipher,
            value="cardan",
            command=self._update_ui,
        ).pack(side=tk.LEFT, padx=5)

        input_frame = ttk.LabelFrame(self, text="Исходный текст",
padding="10")
        input_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=5)
        self.input_text = scrolledtext.ScrolledText(

```

```

        input_frame, wrap=tk.WORD, height=10
    )
    self.input_text.pack(fill=tk.BOTH, expand=True)
    self.input_text.bind("<Button-3>", self._show_context_menu)

    action_frame = ttk.Frame(self, padding="10")
    action_frame.pack(fill=tk.X, padx=10)
    ttk.Button(action_frame, text="Шифровать",
command=self._process_encrypt).pack(
        side=tk.LEFT, padx=5
    )
    ttk.Button(
        action_frame, text="Дешифровать", command=self._process_decrypt
    ).pack(side=tk.LEFT, padx=5)
    self.visualize_button = ttk.Button(
        action_frame, text="Визуализировать",
command=self._visualize_scytale
    )
    self.visualize_button.pack(side=tk.LEFT, padx=5)
    ttk.Separator(action_frame, orient="vertical").pack(
        side=tk.LEFT, padx=10, fill="y"
    )
    ttk.Button(action_frame, text="ОЧИСТИТЬ ВСЁ",
command=self._clear_all).pack(
        side=tk.LEFT, padx=5
    )

    key_frame = ttk.Frame(self, padding="10")
    key_frame.pack(fill=tk.X, padx=10)
    self.key_label = ttk.Label(key_frame, text="Ключ (число):")
    self.key_entry = ttk.Entry(key_frame, width=10)
    self.cardan_key_label = ttk.Label(key_frame, text="Ключ (слово):")
    self.cardan_key_entry = ttk.Entry(key_frame, width=15)
    self.grille_size_label = ttk.Label(key_frame, text="Размер:")
    self.grille_size_var = tk.IntVar(value=6)
    self.grille_size_spinbox = ttk.Spinbox(
        key_frame,
        from_=4,
        to=20,
        increment=2,
        width=5,
        textvariable=self.grille_size_var,
    )
    self._update_ui()

    output_frame = ttk.LabelFrame(self, text="Результат", padding="10")
    output_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=5)
    self.output_text = scrolledtext.ScrolledText(
        output_frame, wrap=tk.WORD, height=10, state="disabled"
    )
    self.output_text.pack(fill=tk.BOTH, expand=True)
    self.output_text.bind("<Button-3>", self._show_context_menu)

def _update_ui(self):
    is_scytale = self.selected_cipher.get() == "scytale"
    if is_scytale:
        self.visualize_button.pack(side=tk.LEFT, padx=5)
    else:
        self.visualize_button.pack_forget()
    widgets_scytale = [self.key_label, self.key_entry]
    widgets_cardan = [
        self.cardan_key_label,

```

```

        self.cardan_key_entry,
        self.grille_size_label,
        self.grille_size_spinbox,
    ]
    for w in widgets_cardan if is_scytale else widgets_scytale:
        w.pack_forget()
    for w in widgets_scytale if is_scytale else widgets_cardan:
        w.pack(side=tk.LEFT, padx=5)

def _clear_all(self):
    self.input_text.delete("1.0", tk.END)
    self.output_text.config(state="normal")
    self.output_text.delete("1.0", tk.END)
    self.output_text.config(state="disabled")

def _display_result(self, text):
    self.output_text.config(state="normal")
    self.output_text.delete("1.0", tk.END)
    self.output_text.insert("1.0", text)
    self.output_text.config(state="disabled")

def _load_scytale_example(self):
    self.selected_cipher.set("scytale")
    self._update_ui()
    self._clear_all()
    self.input_text.insert("1.0", "ЭТО ШИФР ДРЕВНЕЙ СПАРТЫ")
    self.key_entry.delete(0, tk.END)
    self.key_entry.insert(0, "4")

def _load_cardan_example(self):
    self.selected_cipher.set("cardan")
    self._update_ui()
    self._clear_all()
    self.input_text.insert("1.0", "Это простой текст для демонстрации")
    self.cardan_key_entry.delete(0, tk.END)
    self.cardan_key_entry.insert(0, "secret")
    self.grille_size_var.set(6)

def _show_scytale_help(self):
    messagebox.showinfo(
        "Шифр Скитала (Scytale)",
        "Принцип: Текст записывается на ленту, намотанную на цилиндр. Затем лента разматывается.\n\nКлюч: Диаметр цилиндра (целое число).\n\nКриптостойкость: Крайне низка.",
    )

def _show_cardan_help(self):
    messagebox.showinfo(
        "Решётка Кардано (Cardan Grille)",
        "Принцип: Текст вписывается через вырезы в решётке, которая поворачивается 4 раза. Решётка генерируется на основе ключевого слова.\n\nКлюч: Ключевое слово и размер решётки.\n\nОграничения: Текст дополняется символом '_' до размера N*N.",
    )

def _open_file(self):
    filepath = filedialog.askopenfilename(
        filetypes=[("Text Files", "*.txt"), ("All files", "*.*")]
    )
    if not filepath:
        return
    try:

```

```

        with open(filepath, "r", encoding="utf-8") as f:
            self.input_text.delete("1.0", tk.END)
            self.input_text.insert("1.0", f.read())
    except Exception as e:
        messagebox.showerror("Ошибка чтения файла", str(e))

def _save_file(self):
    content = self.output_text.get("1.0", tk.END).rstrip("\n")
    if not content:
        messagebox.showwarning("Предупреждение", "Поле результата
пусто.")
    return
    filepath = filedialog.asksaveasfilename(
        defaultextension=".txt",
        filetypes=[("Text Files", "*.txt"), ("All files", "*.*")],
    )
    if not filepath:
        return
    try:
        with open(filepath, "w", encoding="utf-8") as f:
            f.write(content)
    except Exception as e:
        messagebox.showerror("Ошибка сохранения файла", str(e))

def _show_grille_visualization(self, grille):
    vis_window = tk.Toplevel(self)
    vis_window.title("Сгенерированная решётка Кардано")
    vis_window.transient(self)
    vis_window.resizable(False, False)
    vis_window.configure(bg=self.COLORS["bg"])

    grille_str = "\n".join(
        " ".join(["■" if cell else "□" for cell in row]) for row in
grille
    )

    ttk.Label(
        vis_window,
        text=grille_str,
        font=("Consolas", 16),
        padding=20,
        background=self.COLORS["bg"],
        foreground=self.COLORS["fg"],
    ).pack()

def _visualize_scytale(self):
    try:
        text = self.input_text.get("1.0", tk.END).rstrip("\n")
        key = int(self.key_entry.get())
        if not text or key <= 0:
            raise ValueError("Текст не пуст, ключ > 0.")
        ScytaleAnimationWindow(self, text, key, self.COLORS)
    except Exception as e:
        messagebox.showerror("Ошибка ввода", f"Невозможно запустить
анимацию: {e}")

def _process_encrypt(self):
    # Корректное получение текста из виджета.
    text = self.input_text.get("1.0", tk.END).rstrip("\n")
    if not text:
        messagebox.showerror("Ошибка", "Исходный текст пуст.")
    return

```

```

try:
    if self.selected_cipher.get() == "scytale":
        key = int(self.key_entry.get())
        result = ciphers.scytale_encrypt(text, key)
    else: # cardan
        key_phrase = self.cardan_key_entry.get()
        if not key_phrase:
            messagebox.showerror("Ошибка", "Введите ключевое слово.")
            return
        random.seed(key_phrase)
        size = self.grille_size_var.get()
        grille = ciphers.cardan_generate_grille(size)
        self._show_grille_visualization(grille)
        padded = text.ljust(size * size, "_")[: size * size]
        result = ciphers.cardan_encrypt(padded, grille)
        self._display_result(result)
except Exception as e:
    messagebox.showerror("Ошибка ввода", str(e))

def _process_decrypt(self):
    text = self.input_text.get("1.0", tk.END).rstrip("\n")
    if not text:
        messagebox.showerror("Ошибка", "Исходный текст пуст.")
        return
    try:
        if self.selected_cipher.get() == "scytale":
            key = int(self.key_entry.get())
            result = ciphers.scytale_decrypt(text, key)
        else: # cardan
            key_phrase = self.cardan_key_entry.get()
            if not key_phrase:
                messagebox.showerror("Ошибка", "Введите ключ.")
                return
            # Повторное использование того же seed для восстановления
            random.seed(key_phrase)
            size = self.grille_size_var.get()
            grille = ciphers.cardan_generate_grille(size)
            self._show_grille_visualization(grille)
            if len(text) != size * size:
                messagebox.showerror(
                    "Ошибка", f"Длина текста ({len(text)}) должна быть
{size*size}."
                )
            return
        result = ciphers.cardan_decrypt(text, grille).rstrip("_")
        self._display_result(result)
    except Exception as e:
        messagebox.showerror("Ошибка ввода", str(e))

if __name__ == "__main__":
    app = CipherApp()
    app.mainloop()

```